CHAPTER 1

INTRODUCTION

CanvasCraft is a website for managing the art market, bringing artists, buyers, and galleries together in one place. Discover artwork, connect with creators, and make every art transaction easier.

1.1 PERSPECTIVE

The growth of digital technology has transformed the way products and services are presented, discovered, and purchased. The art industry, however, continues to face challenges in providing independent artists with affordable access to potential buyers. Traditional galleries may involve geographical restrictions, intermediary commissions, and limited exhibition opportunities for emerging creators. (SDG9)

General-purpose e-commerce platforms provide online selling facilities but are not always designed to represent the distinctive characteristics of fine art, such as artistic style, medium, dimensions, creator biography, and the background of an artwork. (SDG8)

The platform allows users to browse curated artworks, view artist portfolios, search and filter listings, and place orders. Artists can submit their work for approval and manage their portfolios after receiving the necessary permissions. Administrators oversee submissions, user accounts, artwork listings, and orders. (SDG9)

CanvasCraft also incorporates direct artist payment information through UPI QR codes and transaction reference recording.

1.2 OBJECTIVE

The primary objective of CanvasCraft is to develop a web-based fine art gallery and creator marketplace that simplifies artwork discovery and supports direct interaction between artists and customers.

The specific objectives are:

1. To develop a secure registration and login system with role-based access control.
2. To provide an interactive gallery for discovering and exploring artworks.
3. To enable customers to submit artwork for administrative review.
4. To implement an approval workflow for promoting eligible users to artist status.
5. To support direct artist payments using UPI payment information.

6. To develop an Artist Studio Dashboard for managing artworks and monitoring sales.

1.3 SCOPE

The scope of CanvasCraft includes the development of frontend and backend components for a digital art marketplace.

1. The system provides customer registration, authentication, artwork discovery, category filtering, search, sorting, artist profiles, artwork submission, administrative approval, direct payment information, order placement, and order tracking.
2. Artists can manage artwork details, update pricing, monitor engagement, maintain payment information, and update fulfillment statuses. Administrators can review submissions, manage user roles, moderate artwork, and oversee orders.
3. The current implementation records payment references and supports the display of artist payment details. Automated bank settlement, commercial courier API integration, augmented reality artwork previews, and advanced AI-based recommendations are identified as possible future enhancements.

CHAPTER 2

REQUIREMENT DESCRIPTION

2.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that CanvasCraft must perform.

Attribute	Requirement
User Authentication and Authorization	The system shall allow users to register and log in using their credentials. Users shall be able to authenticate using their username or email address. Protected functions shall be accessible according to the user's assigned role.
Artwork Gallery and Discovery	The system shall display approved artworks with titles, images, descriptions, prices, dimensions, categories, and artist information. Users shall be able to search, filter, sort, and explore the available listings.
Artist Onboarding and Approval	Registered users shall be able to submit artworks with the required details and images. New submissions shall enter a pending review queue. Administrators shall approve or reject submissions. Approval shall publish the artwork and promote eligible users to artist status.
Order Placement and Payments	Customers shall be able to enter delivery details and place orders. The checkout interface shall display the artist's payment information where applicable. Buyers shall be able to record their UPI transaction reference and optionally submit a payment receipt.
Administrator Management	Administrators shall be able to review artwork submissions, manage customers and artists, moderate listings, oversee orders, and view platform-level statistics.

2.2 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe the quality attributes and platform characteristics expected of CanvasCraft.

Attribute	Requirement
Performance	The application should provide responsive navigation, efficient API requests, and timely loading of artwork listings.
Security	Passwords should be securely hashed. Authentication tokens and role-based authorization should protect restricted operations.
Usability	The interface should provide clear navigation, readable artwork information, and consistent interaction patterns.
Reliability	The persistence layer should support fallback operation when the primary database is unavailable, subject to the implementation's capabilities.
Portability	The application should support suitable development environments with the required runtime, dependencies, and browser.
Scalability	The modular architecture should support future improvements to artwork discovery, payments, delivery integration, and artist services.

CHAPTER 3

SYSTEM DESIGN

3.1 ARCHITECTURE DESIGN

CanvasCraft follows a client-server architecture based on the MERN stack. The React.js frontend provides the user interface, while the Node.js and Express.js backend processes requests, enforces business rules, and communicates with the data persistence layer.

The system architecture consists of the following layers:

1. **Presentation Layer (Desktop UI)**

This layer provides the screens users interact with: login and signup, dashboards, artwork posting forms, and order forms. It captures user input and sends actions to the business logic layer.

2. **Business Logic Layer (Application Core)**

This layer processes user actions and applies the application's rules. It includes authentication and verification, transaction management, artwork uploads, and like counting. The Account Management Engine manages account-related data and generates the order and artwork post datasets.

3. **Data Access Layer**

This layer handles communication between the application and its data stores. The Database Provider Engine is shown as the MongoDB access component; the layer also performs CRUD operations on a local encrypted database file.

4. **Database Layer**

The diagram shows MongoDB as the database that stores application data. Data queries and processes flow from the business logic layer through the Data Access Layer to the database, with local CRUD operations also shown beneath the provider engine.

The user interacts with the frontend, which sends requests to the backend API. The backend validates requests, verifies authentication and authorization, processes the required operation, and accesses the persistence layer. The resulting response is returned to the frontend and displayed to the user.

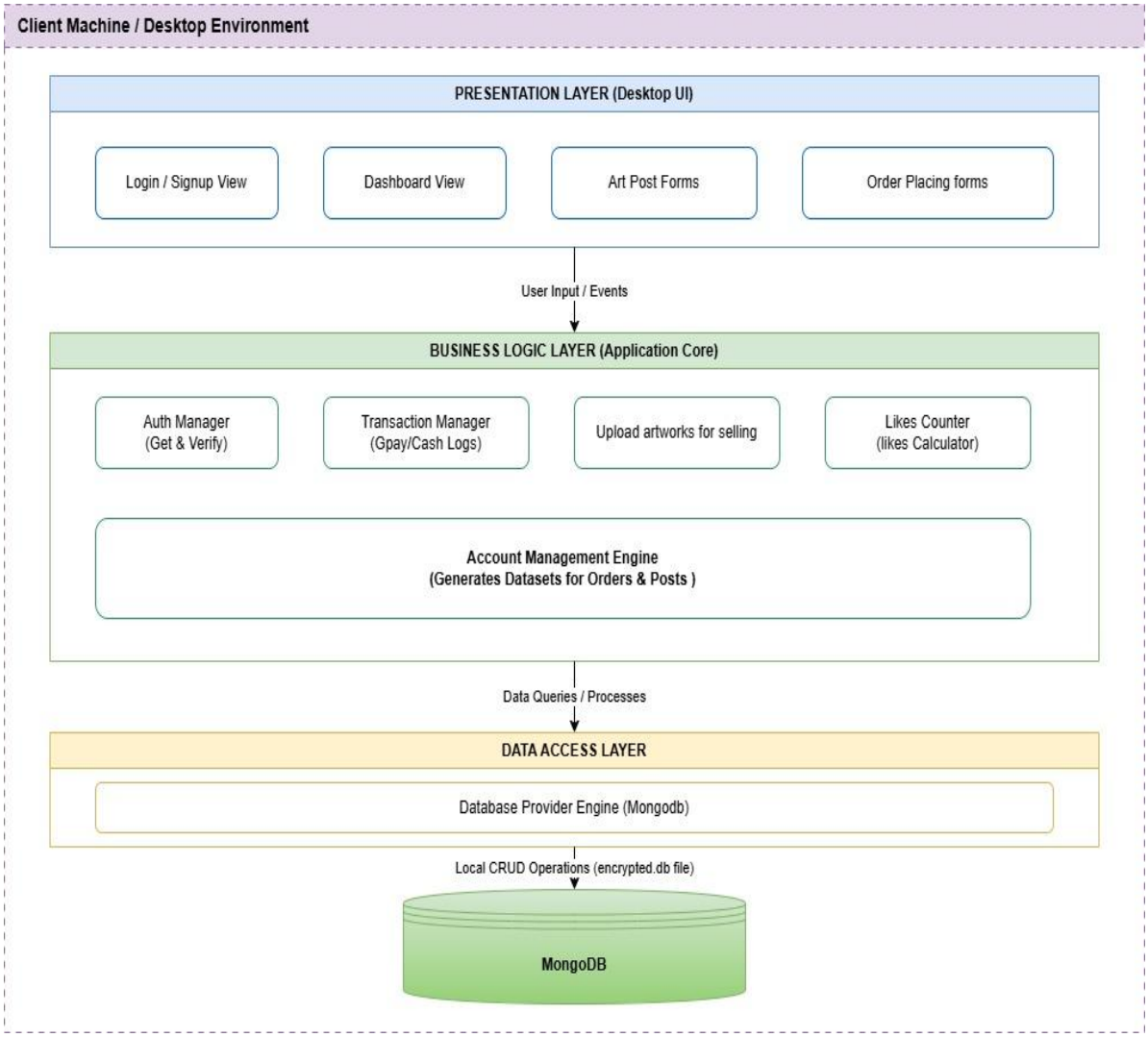


Figure 3.1: CanvasCraft System Architecture Diagram

3.2 UML DIAGRAMS:

The figure3.2 shows the use case diagram represents the ArtMarket Management system lets admins manage users, verify artists, oversee artworks and orders, and browse the gallery. Guests can explore and order art or apply to become artists, while artists manage their profiles and listings and view orders.

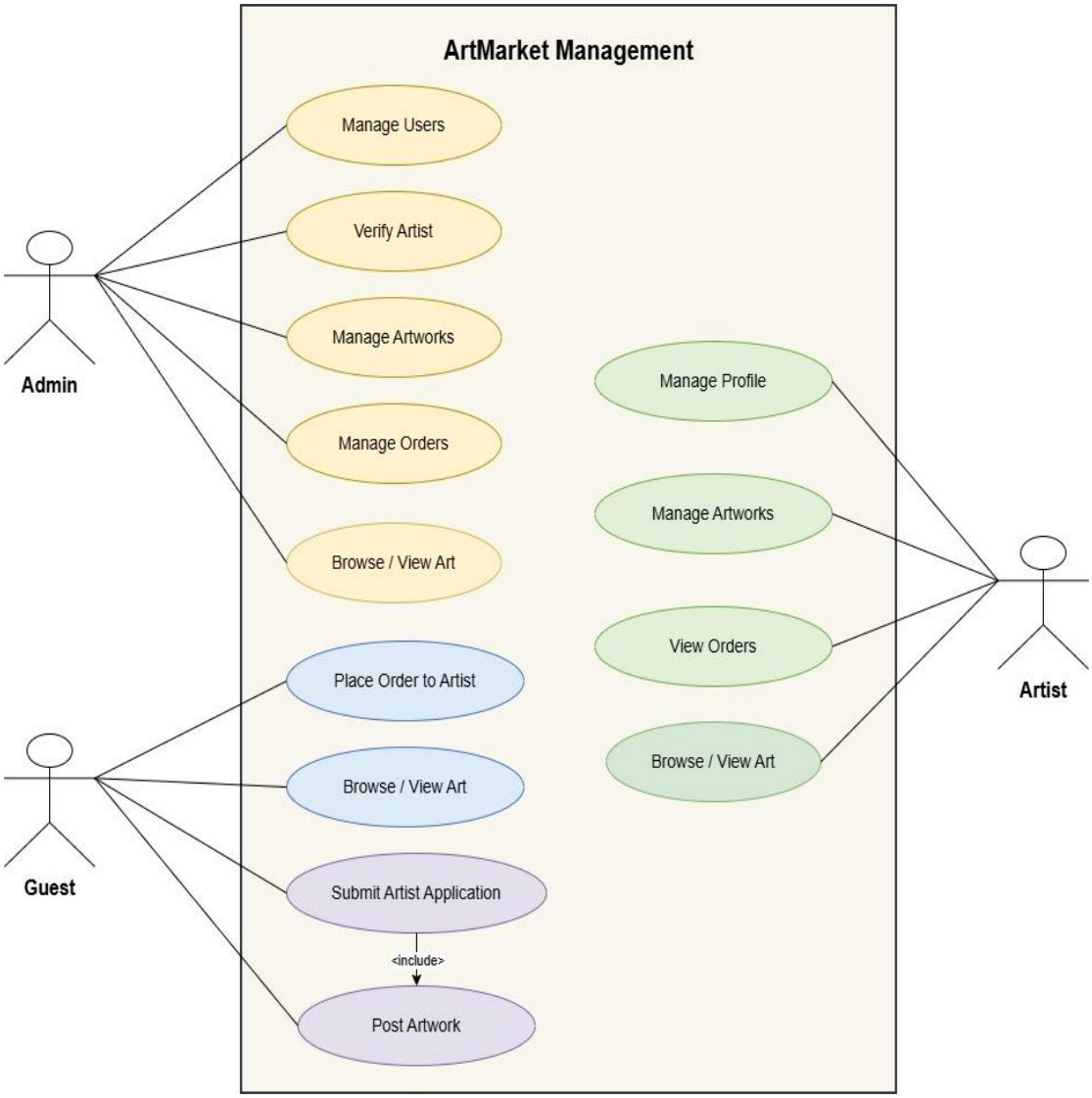


Figure 3.2: Usecase Diagram

The figure 3.3 shows the sequence diagram shows how a guest interacts with the ArtMarket Management system to register or log in, browse artwork, and place an order. It also illustrates how the guest submits an artist application for admin review before logging out.

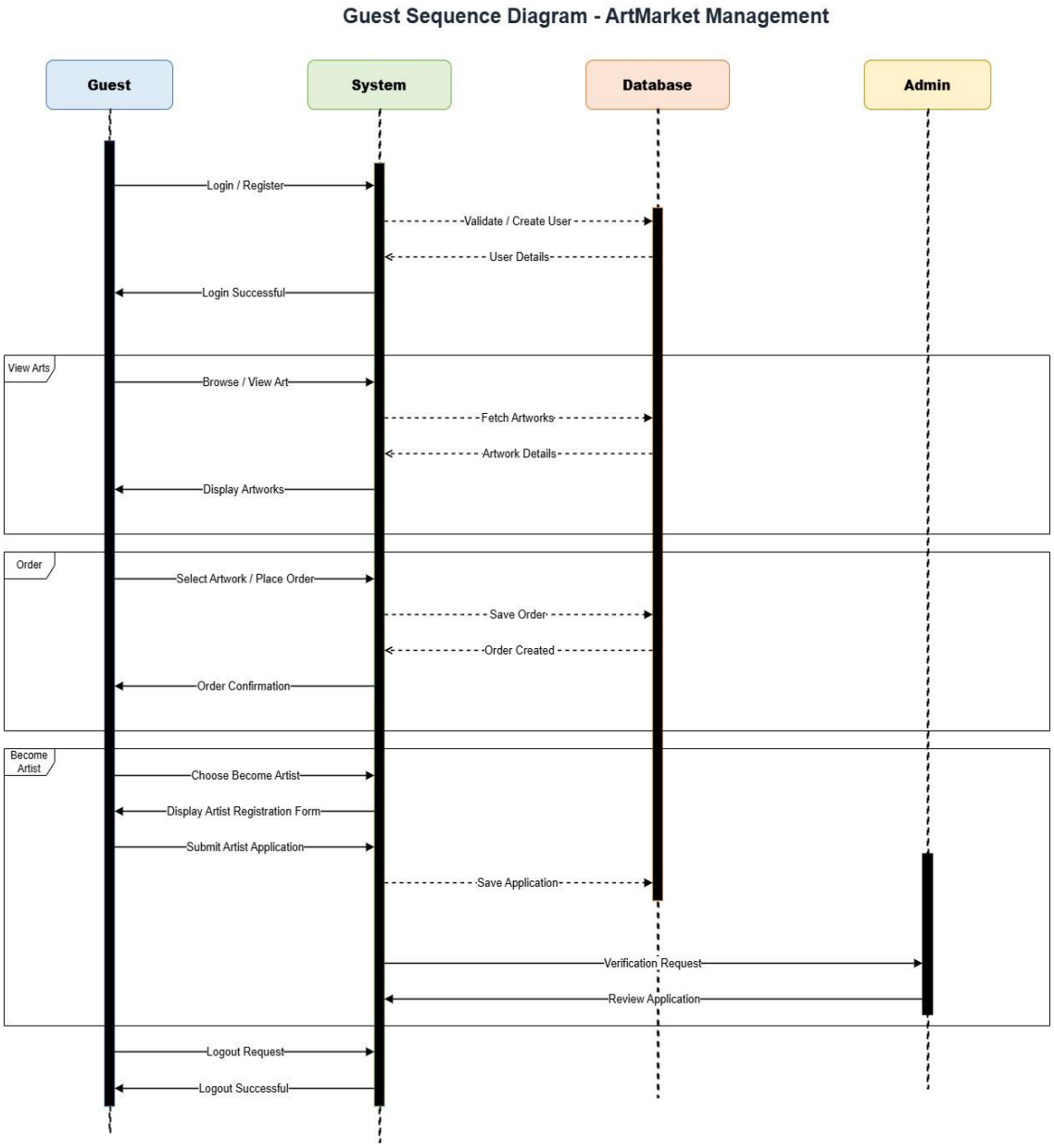


Figure 3.3 Sequence Diagram

The figure 3.4 shows the communtication diagram represents the artwork submission process: a user uploads artwork and enters its details through the art upload form, with a QR code generated as part of the flow. The admin reviews the submission and records the decision; approved artwork is published, while rejected submissions trigger a notification.

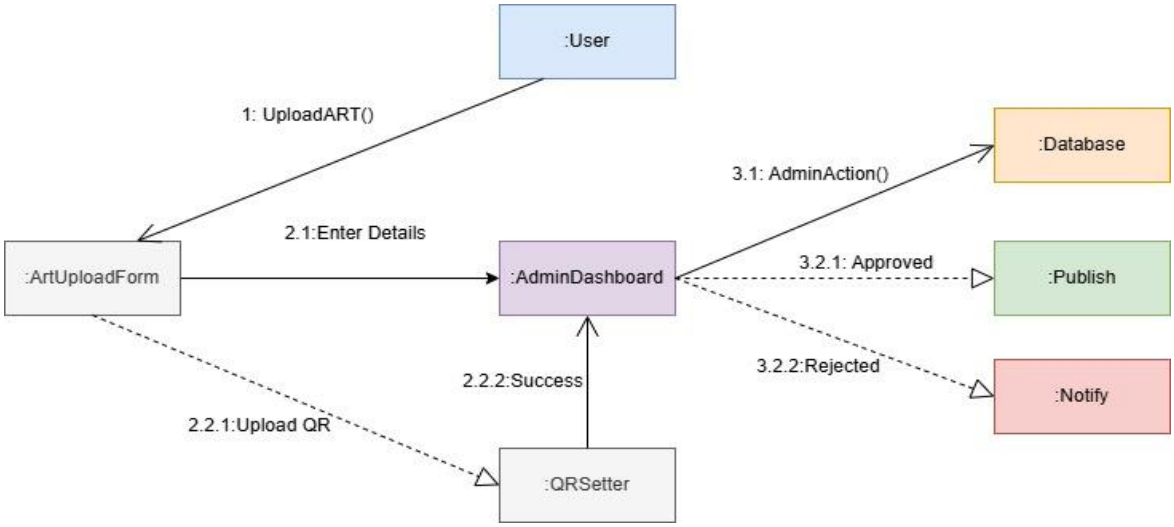


Figure 3.4: Communication Diagram

The figure 3.5 is the activity diagram which shows how users choose a role after authenticating and follow the corresponding workflow. Guests can browse and order artwork or request artist status; admins review requests, and approved users proceed to the artist dashboard, while rejected applicants are notified.

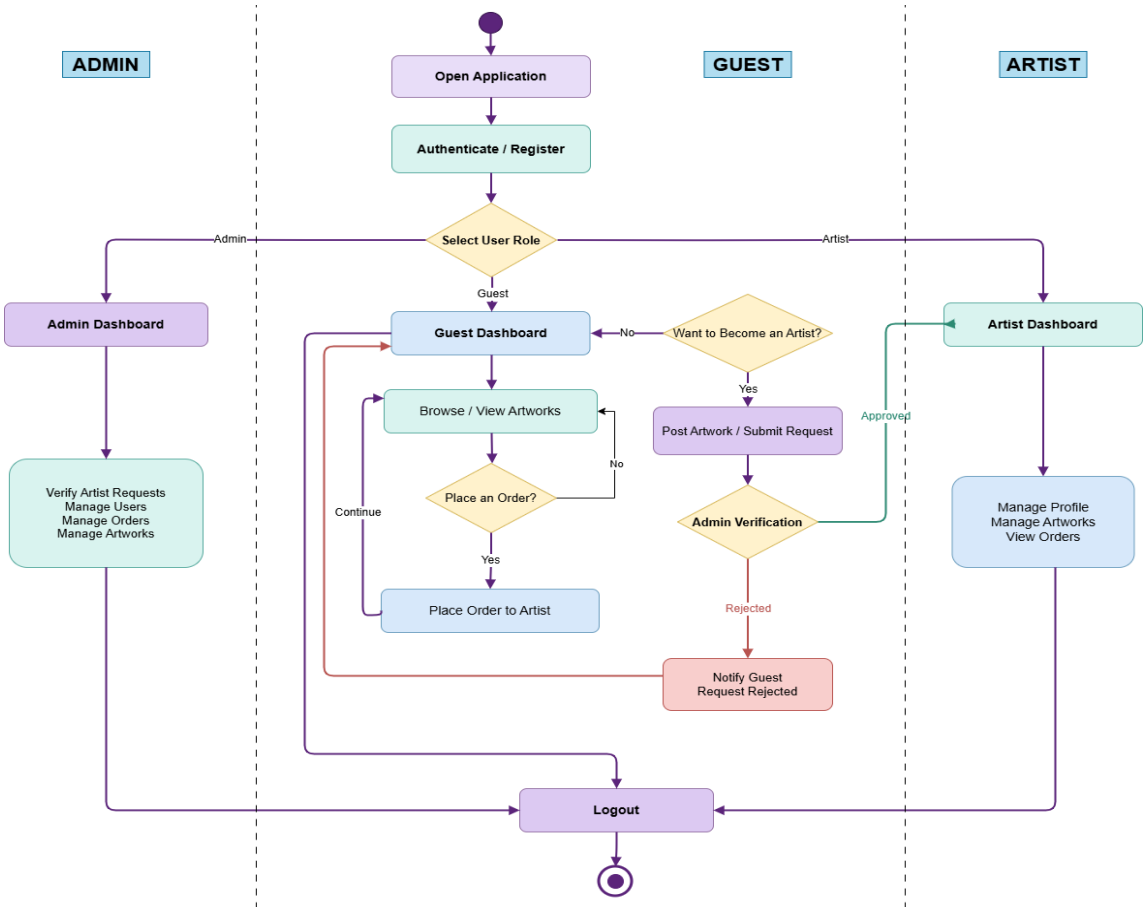


Figure 3.5 Activity Diagram

3.3 DESIGN COMPONENTS

3.3.1 Frontend:

The frontend is developed using React.js and JavaScript. Bootstrap 5 and CSS3 are used to create a responsive interface with a dark-themed Glassmorphism design.

The frontend includes the public gallery, authentication pages, artwork detail views, artist profiles, checkout forms, customer-facing order information, the Artist Studio Dashboard, and the administrator interface.

3.3.2 Backend:

The backend uses Node.js and Express.js. Controllers handle authentication, artwork management, order processing, and administrative functions. Also it supports authentication, role verification, request processing, and file uploads.

3.4 DATABASE DESCRIPTION

The principal data entities in CanvasCraft are User, Art, and Order. Listed below is a description of the database schemas used for CanvasCraft.

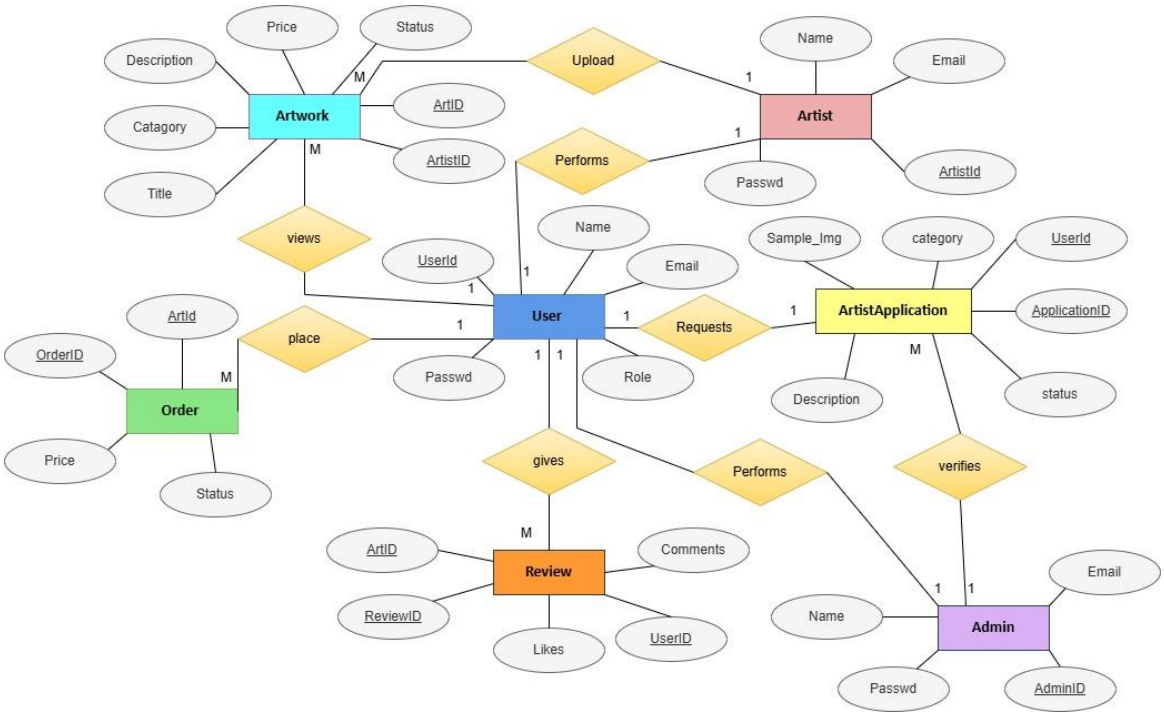


Figure 3.6: ER Diagram

3.4.1 User Structure

As shown in Table 3.1, the User entity stores user identity, contact information, authentication details, role information, artist profile data, and payment details where applicable.

Attribute Name	Type	Constraint(s)	Description
Username	String	Required	Unique username of the user
Email	String	Required	Email address used for login and contact
Password	String	Required	Hashed password used for authentication
Role	String		Customer, artist, or administrator role
Artist Profile	Object		Artist biography and portfolio information
Payment Details	Object		Artist UPI ID and UPI QR code for direct payments

Table 3.1: User Entity / Collection Schema Definition

3.4.2 Art Structure

As shown in Table 3.2, the Art entity stores the details of each artwork and its review status.

Attribute Name	Type	Constraint(s)	Description
Title	String	Required	Title of the artwork
Category	String	Required	Artistic medium or category
Description	String	Required	Description and background of the artwork
Dimensions	String	Required	Physical dimensions of the artwork
Price	Number	Required	Listed price of the artwork
Image	String	Required	Uploaded image information
Creator	ObjectId		Reference to the creating user
Status	String		Approval status (pending, approved, rejected)
Likes	Number		Engagement information

Table 3.2: Art Entity / Collection Schema Definition

3.4.3 Order Structure

As shown in Table 3.3, the Order entity stores the details of every order placed on the platform.

Attribute Name	Type	Constraint(s)	Description
Order ID	ObjectId		Unique identifier of the order
Customer	ObjectId		Reference to the customer
Artist	ObjectId		Reference to the artist

Attribute Name	Type	Constraint(s)	Description
Artwork	ObjectId		Reference to the ordered artwork
Delivery Details	Object	Required	Delivery information supplied by the customer
Payment Reference	String	Required	UPI transaction reference recorded by the buyer
Fulfillment Status	String		Current fulfillment status of the order

Table 3.3: Order Entity / Collection Schema Definition

3.5 LOW LEVEL DESIGN

The following section illustrates the internal operation of the main modules of the application.

3.5.1 Authentication

Validates user credentials, verifies passwords, issues authentication tokens, and restricts protected operations.

3.5.2 Artwork Management

Handles artwork submission, retrieval, editing, deletion, filtering, and administrative approval.

3.5.3 Order Management

Records customer orders, links orders to the relevant artwork and artist, stores delivery and payment reference information, and supports fulfillment updates.

3.5.4 Administrative Management

Provides controlled access to artwork moderation, user management, order oversight, and platform statistics.

This chapter described the architecture, design components, database entities, low-level module design, and user interfaces of CanvasCraft. The modular layered design separates presentation, business logic, and data persistence, which simplifies development and future extension of the system.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 IMPLEMENTATION OVERVIEW

CanvasCraft is implemented as a full-stack web application with separate frontend and backend components. The frontend manages presentation and user interactions, while the backend processes requests and applies authentication, authorization, business logic, and data operations.

The application is organized into modules for user management, artwork discovery, artist onboarding, order management, artist operations, and administrative control.

4.2 USER INTERFACE DESIGN

The CanvasCraft interface is designed to provide an organized browsing and management experience. The major interfaces include:

1. Landing page and featured artwork showcase.
2. Gallery page with search, category filtering, and sorting.
3. Artwork details and artist information page.
4. User registration and login pages.
5. Artwork submission and administrative review pages.
6. Order placement and payment information interface.
7. Artist Studio Dashboard.
8. Administrator Control Center.

4.2.1 Landing Page

The Figure 4.1 shows the Landing Page and Featured Artwork Showcase.

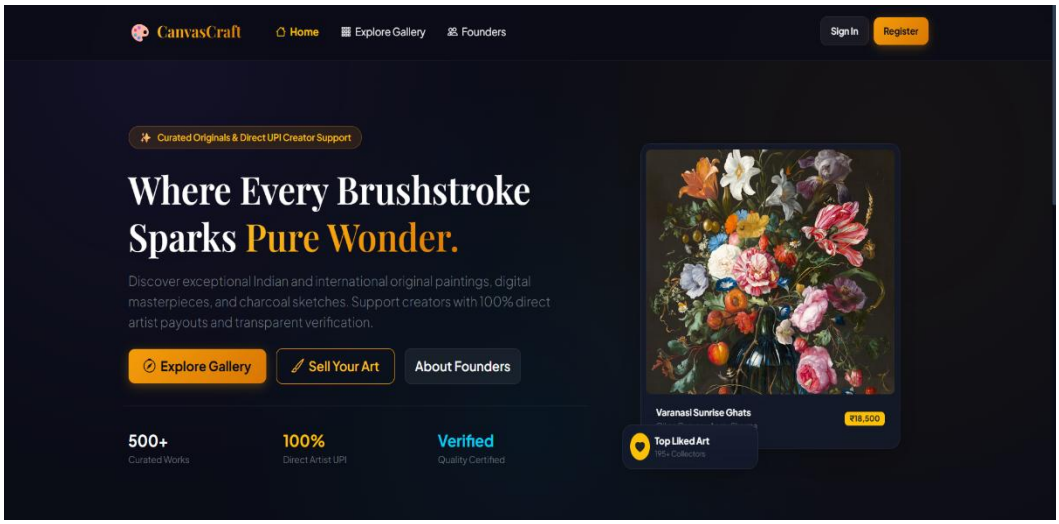


Figure 4.1: Landing Page screenshot

4.2.2 Gallery

The Figure 4.2 shows the Gallery with Search, Category Filtering and Sorting.

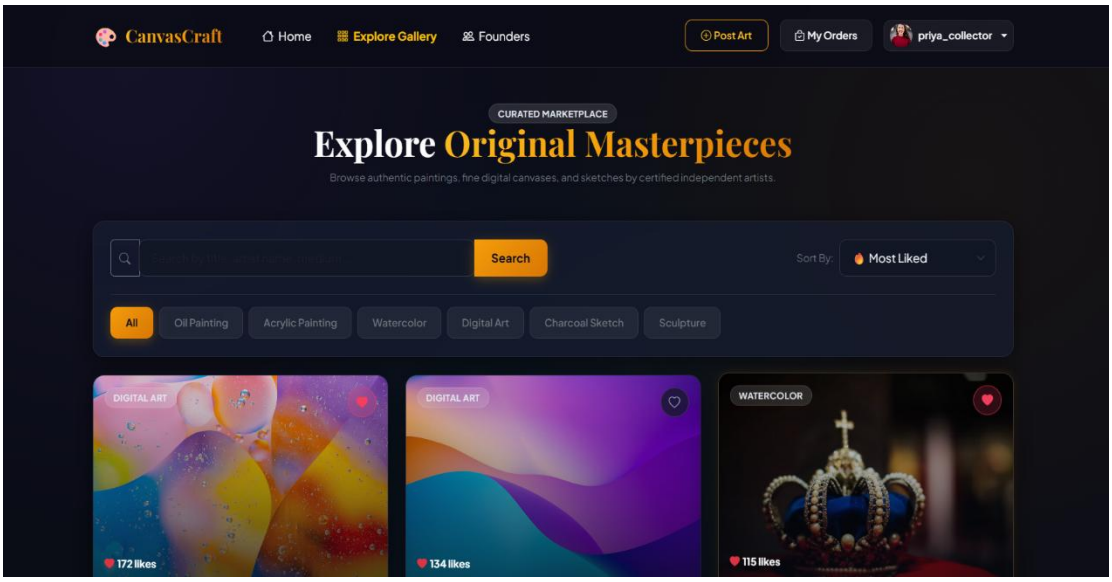


Figure 4.2: Gallery screenshot

4.2.3 Artwork Details and Artist Profile

The Figure 4.3 shows the Artwork Details and Artist Information Page.

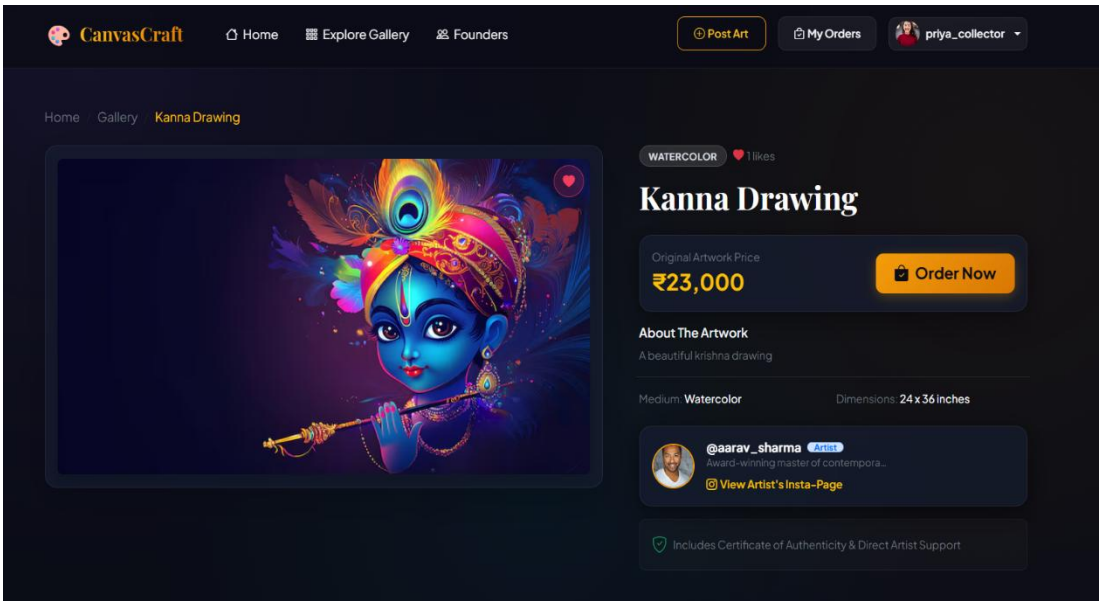


Figure 4.3: Artwork Details and Artist Information Page

4.2.4 Login Page

The Figure 4.4 shows the User Login Page.

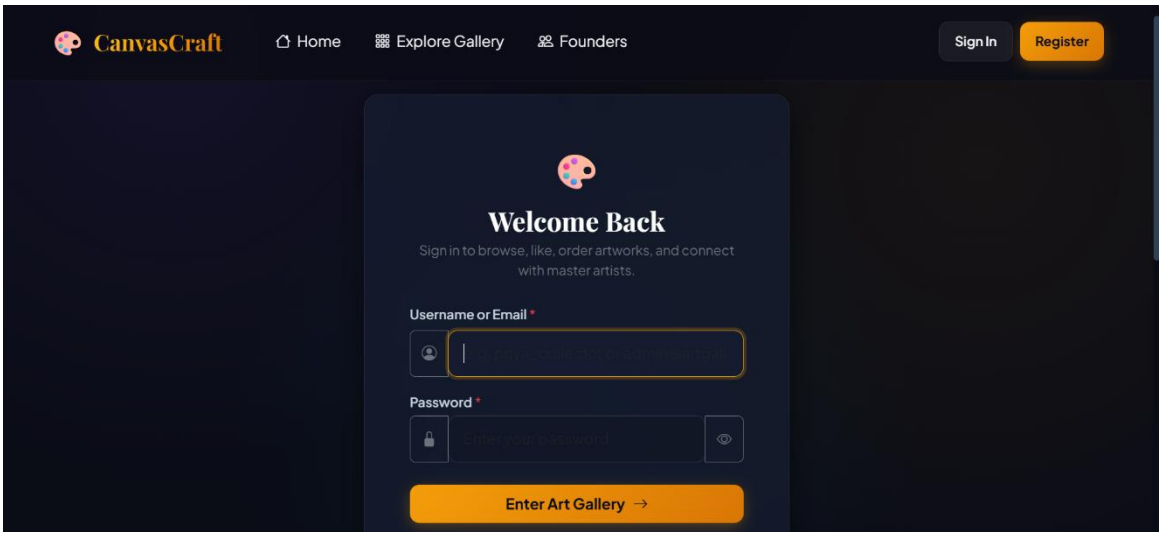


Figure 4.4: User Login Page screenshot

4.2.5 Artwork Submission and Review

The Figure 4.5 shows the Artwork Submission and Administrative Review Pages.

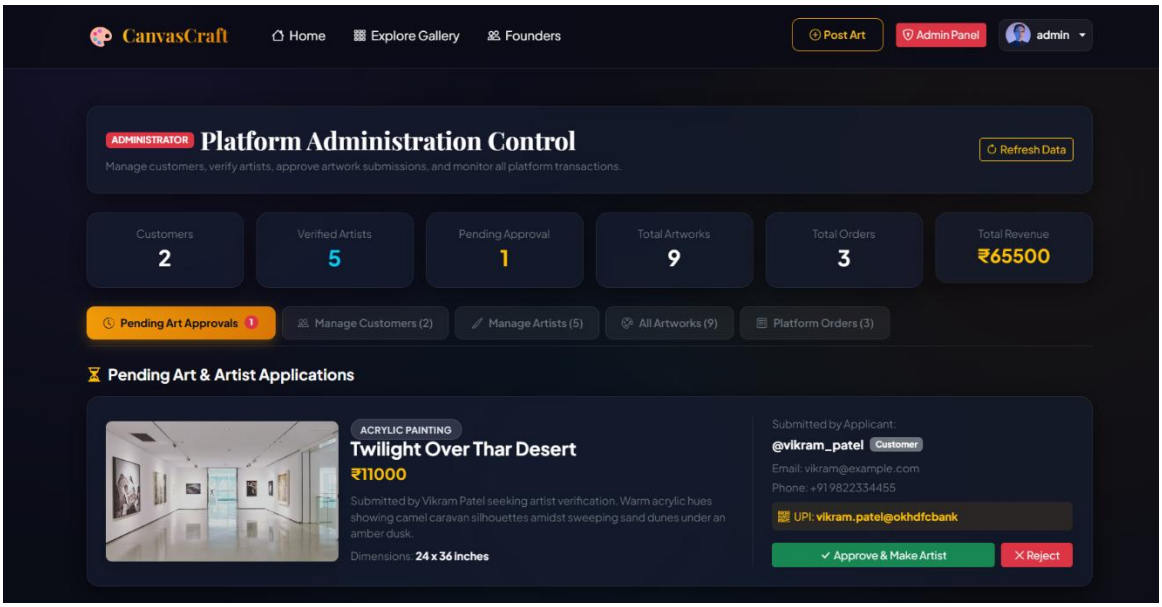


Figure 4.5: Artwork Submission and Administrative Review Page

4.2.6 Order Placement and Payment

The Figure 4.6 shows the Order Placement and Artist Payment Interface.

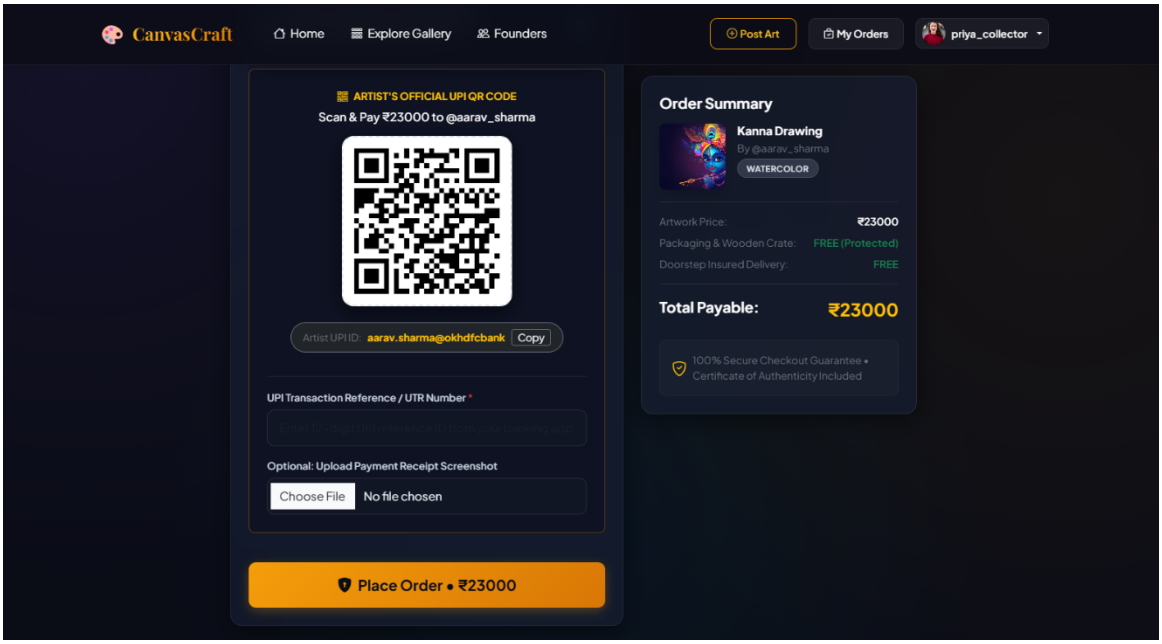


Figure 4.6: Order Placement and Artist Payment Interface

4.2.7 Artist Studio Dashboard

The Figure 4.7 shows the Artist Studio Dashboard.

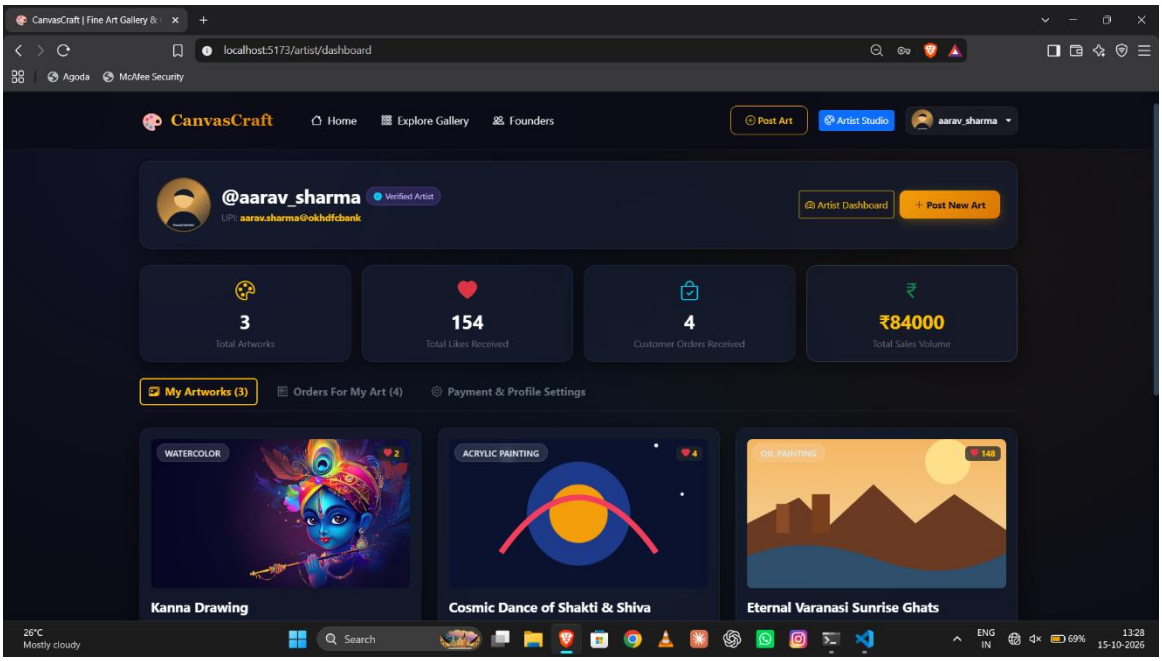


Figure 4.7: Artist Studio Dashboard

4.2.8 Administrator Control Center

The Figure 4.8 shows the Administrator Control Center.

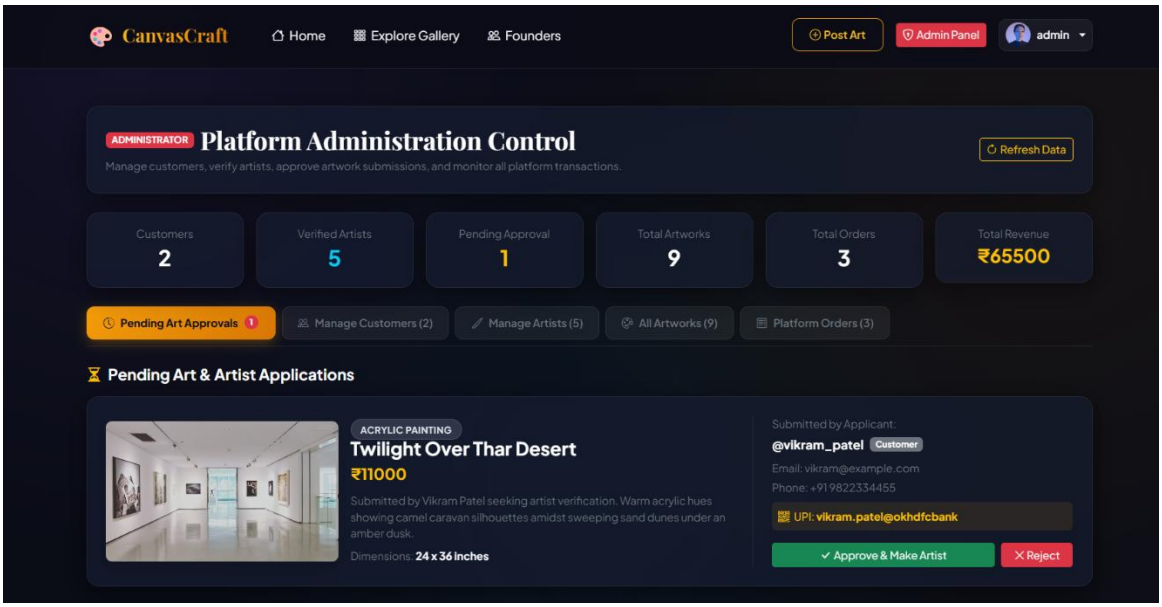


Figure 4.8: Administrator Control Center

4.3 DATABASE AND UNIFIED PERSISTENCE IMPLEMENTATION

The unified Data Access Layer separates application operations from the underlying persistence engine. MongoDB is used through Mongoose when available. The local JSON store provides a fallback mechanism for supported operations.

This design reduces dependence on a single database configuration and makes the project more convenient to run in development and academic evaluation environments.

4.4 AUTHENTICATION AND AUTHORIZATION SECURITY

CanvasCraft uses password hashing and token-based authentication to protect user accounts. Role-based authorization restricts artist and administrator operations to permitted users. Input validation and controlled file handling provide additional protection for requests involving user data and artwork uploads.

Security-sensitive operations are verified on the server rather than relying solely on frontend restrictions.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 TEST CASES AND RESULTS

5.1.1 Test Cases and Results for Login function:

The Table 5.1, Table 5.2 shows that the possible test data for the both positive and negative test case given below, if the user is already having account then the output is true otherwise false.

Table 5.1: Positive Test Case and result for Login

Test Case ID	TC1
Test Case Description	It tests whether the given login details are valid or not
Test Data	Registered username/email and correct password
Expected Output	TRUE
Result	PASS

Table 5.2: Negative Test Case and result for Login

Test Case ID	TC2
Test Case Description	It tests whether the given login details are valid or not
Test Data	Invalid username/email or incorrect password
Expected Output	FALSE
Result	PASS

5.1.2 Test Cases and Results for Signup Functionality

These test cases evaluate whether the system accepts valid registration details and handles invalid registration information appropriately.

Table 5.2: Positive Test Case for Signup

Test Case ID	TC3
Test Case Description	Verifies whether a new user can register with valid details.
Test Data	Valid username, email, password, and role
Expected Output	User details stored successfully
Result	PASS

Table 5.4: Negative Test Case for Signup

Test Case ID	TC4
Test Case Description	Verifies whether the system rejects invalid or incomplete registration details.
Test Data	Missing or invalid registration fields
Expected Output	Registration rejected and an error message displayed
Result	PASS

5.2 VERIFICATION TEST CASES

The project includes an end-to-end test intended to verify the application's principal workflows. The tests cover valid and invalid authentication, protected routes, artwork retrieval, artwork submission, administrative approval, order creation, fulfillment updates, and persistence operations.

Table 5.5: Test Case Format

Test Case ID	Function	Expected Result	Actual Result
TC-01	Valid login	User authenticated successfully	Record actual result
TC-02	Invalid login	Authentication rejected	Record actual result
TC-03	Artwork listing	Approved artwork displayed	Record actual result
TC-04	Unauthorized administrative access	Access denied	Record actual result
TC-05	Artwork submission	Submission recorded with appropriate status	Record actual result
TC-06	Administrative approval	Approved artwork published	Record actual result
TC-07	Order creation	Order recorded with relevant details	Record actual result
TC-08	Order status update	Updated status displayed	Record actual result
TC-09	Artwork filtering	Matching artwork displayed	Record actual result
TC-10	Protected artist operation	Access determined by authorization	Record actual result
TC-11	User management	Authorized changes processed	Record actual result
TC-12	Persistence fallback	Supported fallback operation completed	Record actual result

5.3 EXPECTED AND ACTUAL RESULTS

The expected outcome is a functional marketplace that provides artwork discovery, secure account access, artist onboarding, administrative moderation, order placement, and fulfillment tracking.

The actual results are documented using the outputs of the implemented application and the automated test execution, including observed errors, successful operations, and any limitations encountered during testing.

5.4 DISCUSSION OF RESULTS

CanvasCraft combines gallery presentation, creator onboarding, artwork moderation, direct artist payment information, and order tracking in a single application. Its modular architecture separates frontend interactions from backend processing and data persistence.

The dual-engine persistence approach provides a fallback option for supported database operations. The artist and administrator dashboards offer dedicated interfaces for creator management and platform governance.

5.5 LIMITATIONS AND OBSERVATIONS

The current design records payment references but does not itself establish that a bank transfer has been settled. Automated bank webhooks and escrow services require further integration. Third-party courier tracking is also a future enhancement. Advanced visual recommendations, augmented reality previews, and live art auctions are not part of the stated current scope.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT(S)

6.1 CONCLUSION

CanvasCraft is a full-stack web-based fine art gallery and creator marketplace developed using the MERN stack. It provides an integrated environment for artwork discovery, artist portfolio presentation, creator onboarding, administrative review, order management, and direct artist payment information. (SDG9)

The application incorporates role-based access control, artwork filtering, artist dashboards, order tracking, and a unified persistence layer supporting MongoDB and local JSON storage. Its modular architecture allows the major features to be developed and maintained separately. The system allows everyone without any inequality which promoting efficient art market management and responsible economic growth. (SDG 10)

The project demonstrates how web technologies can be used to create a specialized marketplace for fine art and independent creators. Further testing and future enhancements can improve its payment verification, delivery management, and personalized discovery capabilities.

6.2 PROJECT ACHIEVEMENTS

The principal project objectives achieved are:

1. Developing a web-based fine art gallery using the MERN stack.
2. Implementing customer, artist, and administrator roles.
3. Providing artwork discovery and portfolio features.
4. Establishing an artwork submission and curation workflow.
5. Supporting order management and fulfillment tracking.
6. Integrating direct artist payment information.

6.3 CURRENT LIMITATIONS

The system relies on recorded payment references rather than automatic bank settlement confirmation. Delivery updates depend on the implemented order management workflow rather than a complete third-party logistics integration. The local JSON fallback also has operational limitations compared with a production-grade database deployment.

6.4 FUTURE ENHANCEMENTS

Potential future enhancements include:

1. **Augmented Reality Artwork Preview:** Allow buyers to preview artwork placement.
2. **Artwork Authenticity Verification:** Introduce digital certificates of authenticity supported by cryptographic verification.
3. **Automated Payment Reconciliation:** Integrate suitable payment provider webhooks and settlement workflows.
4. **Courier Integration:** Connect to shipping APIs for label generation and delivery tracking.
5. **AI-Powered Recommendations:** Use visual embeddings to recommend artwork based on style, color, and browsing preferences.
6. **Live Virtual Auctions:** Add real-time bidding functionality using WebSocket technology.

APPENDIX – A

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENT:

Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	20 GB available space
Display	1366 × 768 resolution
Network	Internet or local network connectivity as required

SOFTWARE REQUIREMENT:

Operating System	Windows, Linux, or macOS
Frontend	HTML, CSS, JavaScript and React.js
Backend	Node.js and Express.js
Database	MongoDB with Mongoose
Browser	Modern web browser
IDE	Visual Studio Code or compatible IDE
Version Control	Git and GitHub, where used

APPENDIX – B

SOURCE CODE

index.html

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

        <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-size='90'></text></svg>" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

     <meta name="description" content="CanvasCraft - Premium Art Gallery & Creator Marketplace. Discover original paintings, digital artworks, and commission master artists." />

    <title>CanvasCraft | Fine Art Gallery & Creator Marketplace</title>

    <!-- Google Fonts -->

    <link rel="preconnect" href="https://fonts.googleapis.com">

    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

                                                                    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&family=Playfair+Display:ital,wght@0,600;0,700;1,600&display=swap" rel="stylesheet">

    <!-- Bootstrap 5 CSS -->

      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
```

```
  <!-- Bootstrap Icons -->
        <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
 </head>
 <body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
  <!-- Bootstrap 5 JS Bundle -->
                                                                    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
 </body>
</html>
```

server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
const { connectDB } = require('./config/db');
const authRoutes = require('./routes/authRoutes');
const artRoutes = require('./routes/artRoutes');
const orderRoutes = require('./routes/orderRoutes');
```

```
const adminRoutes = require('./routes/adminRoutes');

const app = express();

const PORT = process.env.PORT || 5000;


// Ensure uploads directory exists

const uploadsDir = path.join(__dirname, 'uploads');

if (!fs.existsSync(uploadsDir)) {

  fs.mkdirSync(uploadsDir, { recursive: true });

}


app.use(cors({

  origin: '*',

  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],

  allowedHeaders: ['Content-Type', 'Authorization']

}));


app.use(express.json({ limit: '50mb' }));

app.use(express.urlencoded({ extended: true, limit: '50mb' }));


// Static uploads and local image folders

app.use('/uploads', express.static(uploadsDir));

const localImagesDir = path.join(__dirname, '..', 'images');

const frontendImagesDir = path.join(__dirname, '..', 'frontend', 'public', 'images');
```

```
if (fs.existsSync(localImagesDir)) {
  app.use('/images', express.static(localImagesDir));
}
if (fs.existsSync(frontendImagesDir)) {
  app.use('/images', express.static(frontendImagesDir));
}

// Health check endpoint
app.get('/api/health', (req, res) => {
  res.json({
    status: 'online',
    appName: 'CanvasCraft Art Gallery & Marketplace API',
    timestamp: new Date().toISOString()
  });
});

// Founders endpoint
app.get('/api/founders', (req, res) => {
  res.json({
    success: true,
    founders: [
      {
        name: 'Kamalesh V',
```

```
    title: 'Founder 1',
    role: '3rd year, Computer Science Engineering',
    bio: 'Mepco Schlenk Engineering College, Tamil Nadu, India.',
    image: '/images/Profile image/founder-kamalesh.svg',
      quote: 'Building a space where artists can share their work and reach new audiences.',
    socials: {
      linkedin: 'https://linkedin.com',
      github: 'https://github.com',
      instagram: 'https://instagram.com'
    }
  },
  {
    name: 'SivaKumar S',
    title: 'Founder 2',
    role: '3rd year, Computer Science Engineering',
    bio: 'Mepco Schlenk Engineering College, Tamil Nadu, India.',
    image: '/images/Profile image/founder-sivakumar.svg',
    quote: 'Connecting creators and collectors through a welcoming art marketplace.',
    socials: {
      linkedin: 'https://linkedin.com',
      github: 'https://github.com',
      instagram: 'https://instagram.com'
```

```
        } } ] });
});

// Mount Routes
app.use('/api/auth', authRoutes);
app.use('/api/arts', artRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/admin', adminRoutes);

// Error Handling Middleware
app.use((err, req, res, next) => {
  console.error('Unhandled Server Error:', err);
  res.status(err.status || 500).json({
    success: false,
    message: err.message || 'Internal Server Error'
  });
});

// Start Server & Connect Database
connectDB().then(() => {
  app.listen(PORT, () => {
   console.log(`==============================================`);
```

```
    console.log(` CANVASCRAFT ART GALLERY BACKEND RUNNING ON PORT ${PORT}`);
    console.log(` API Health: http://localhost:${PORT}/api/health`);
    console.log(`================================================`);
  });
});
```

PostArt.jsx

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api/client';

const categories = ['Oil Painting', 'Acrylic', 'Watercolor', 'Digital Art', 'Charcoal Sketch', 'Sculpture'];

export default function PostArt() {
  const { user, isArtist } = useAuth();
  const navigate = useNavigate();
   const [form, setForm] = useState({ title: '', description: '', category: categories[0], dimensions: '', price: '', artistUpiId: user?.upiId || '', artistBio: user?.bio || '' });
  const [artImage, setArtImage] = useState(null);
  const [qrImage, setQrImage] = useState(null);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
```

```
  const [submitting, setSubmitting] = useState(false);


   const update = (event) => setForm((current) => ({ ...current, [event.target.name]: event.target.value }));


  const submit = async (event) => {
    event.preventDefault();
    setError('');
    setSuccess('');
    if (!artImage) return setError('Choose an image for your artwork.');
    if (!isArtist && !form.artistUpiId && !qrImage) return setError('Add your UPI ID or upload a scannable QR image for your first submission.');

    const body = new FormData();
    Object.entries(form).forEach(([key, value]) => body.append(key, value));
    body.append('artImage', artImage);
    if (qrImage) body.append('upiQrImage', qrImage);

    setSubmitting(true);
    try {
          const response = await api.post('/arts', body, { headers: { 'Content-Type': 'multipart/form-data' } });
      setSuccess(response.data.message || 'Artwork submitted.');
```

```
            window.setTimeout(() => navigate(isArtist ? '/artist/dashboard' : '/customer/dashboard'), 1800);
    } catch (requestError) {
      setError(requestError.message || 'Could not submit the artwork.');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="py-5">
      <div className="container" style={{ maxWidth: 820 }}>
        <div className="mb-4">
          <Link to={isArtist ? '/artist/dashboard' : '/customer/dashboard'} className="text-secondary text-decoration-none small"><i className="bi bi-arrow-left me-1" />Back to dashboard</Link>
          <h1 className="text-white fw-bold font-serif mt-2">Share your artwork</h1>
          <p className="text-secondary">Add your artwork details and a local image file.</p>
        </div>

        {error && <div className="alert alert-danger py-2">{error}</div>}
        {success && <div className="alert alert-success py-2">{success}</div>}
```

```
<form onSubmit={submit} className="glass-panel p-4 p-md-5">
 <div className="row g-3">
  <div className="col-md-8">
   <label className="form-label-custom">Artwork title</label>
    <input className="form-control form-control-custom" name="title" value={form.title} onChange={update} required maxLength={100} />
  </div>
  <div className="col-md-4">
   <label className="form-label-custom">Price (₹)</label>
    <input className="form-control form-control-custom" name="price" type="number" min="1" step="1" value={form.price} onChange={update} required />
  </div>
  <div className="col-md-6">
   <label className="form-label-custom">Medium</label>
    <select className="form-select form-control-custom" name="category" value={form.category} onChange={update}>{categories.map((category) => <option key={category}>{category}</option>)}</select>
  </div>
  <div className="col-md-6">
   <label className="form-label-custom">Dimensions</label>
    <input className="form-control form-control-custom" name="dimensions" value={form.dimensions} onChange={update} placeholder="e.g. 24 x 36 inches" />
  </div>
  <div className="col-12">
```

<label className="form-label-custom">Description</label>

<textarea className="form-control form-control-custom" name="description" rows="4" value={form.description} onChange={update} required maxLength={2000} />

<label className="form-label-custom">Artwork image</label>

<input className="form-control form-control-custom" type="file" accept="image/*" onChange={(event) => setArtImage(event.target.files?.[0] || null)} required />

{!isArtist && <>

<label className="form-label-custom">UPI ID</label>

<input className="form-control form-control-custom" name="artistUpiId" value={form.artistUpiId} onChange={update} placeholder="name@bank" />

<label className="form-label-custom">Payment QR image</label>

<input className="form-control form-control-custom" type="file" accept="image/*" onChange={(event) => setQrImage(event.target.files?.[0] || null)} />

Upload a real, scannable QR image. Local sample placeholders cannot receive payments.

```
            <label className="form-label-custom">Short artist bio</label>
              <textarea className="form-control form-control-custom" name="artistBio" rows="2" value={form.artistBio} onChange={update} maxLength={500} />
            </div>
          </>}
        </div>
        <div className="d-flex justify-content-end mt-4">
          <button className="btn btn-gold px-4" disabled={submitting}>{submitting ? 'Submitting…' : 'Submit artwork'} </button>
        </div>
      </form>
    </div>
  </div>
  );
}
```

REFERENCES

1. MongoDB Documentation. https://www.mongodb.com/docs/manual/
2. Mongoose Documentation. https://mongoosejs.com/docs/
3. Express.js Documentation. https://expressjs.com/
4. React Documentation. https://react.dev/
5. Node.js Documentation. https://nodejs.org/docs/
6. Bootstrap Documentation. https://getbootstrap.com/docs/
7. JSON Web Tokens Introduction. https://jwt.io/introduction
8. npm Documentation. https://docs.npmjs.com/